Compiling extended LAMBDA

Into Church's pure lambda calculus

Anay Sharma · CS413, Fall 2026 · Midterm 1

Source ASTs in, pure lambda terms out

Then beta-reduce the terms and decode the answers

The job

Translate numbers, booleans, arithmetic, if, let, pairs, and recursion

Output only variables, single-argument lambdas, and applications

Compile the program. Do not run it and encode the answer

One call-by-value evaluator for source ASTs and target terms

Show that decoded target results match the source results

Source language

The Assignment 5 AST: int, bool, var, lam, app, if, let, pair, fst, snd, fix

Operators: + - * < <= > >= == != and or not

Subtraction is truncated: 2 - 5 is 0

if tests a boolean. and and or short-circuit left to right

let binds only in its body. fix binds both names; if they match, the parameter wins

A negative literal or unknown operator is a compile error, even in dead code

Target language and reduction

Term = Var(x) | Lam(x, body) | App(fn, arg)

Printed as x, (lam x. body), (fn arg)

Weak call-by-value: function, then argument, then substitute

(lam x. body) v → body[x := v], only when v is a lambda

A lambda is finished. Nothing reduces under it

A budget counts beta steps. Running out is exhaustion, not a proof of divergence

Architecture

target.py — terms, printer, capture-avoiding substitution

church.py — closed encodings, expanded into pure terms

compile.py — CompileVisitor. Never imports or calls the evaluator

evaluate.py — one EvaluateVisitor for both languages. Never imports the compiler

- Var, Lam, App share visit_var, visit_lam, visit_app with the source nodes

observe.py — numbers, booleans, pairs, with their own step budget

demo.py — the fourteen programs, saved to demo-out/

Church data

TRUE = lam t. lam f. t FALSE = lam t. lam f. f

NUM(n) = lam s. lam z. s applied n times to z

PAIR = lam a. lam b. lam p. p a b

NUM(0) and FALSE are the same lambda, so the decoder is told the kind

Numbers: run v STEP BASE, then count the spine

Booleans: run v with two closed markers and see which comes back

Bad shape is malformed. Running out of steps is exhaustion

Predecessor of zero is zero

Start from the pair (0, 0)

Each step: (previous, current) becomes (current, current + 1)

After n steps the first component is n - 1

Zero steps leave it at 0

Subtraction applies pred as many times as the second number

So 2 - 5 reduces to 0, and every comparison is built on it

Recursion uses Z, not Y

fix f(x). e compiles to Z (lam f. lam x. C[e])

Z g reduces to a lambda under call-by-value

The recursive copy waits inside lam v. x x v until it gets an argument

Y keeps unfolding before it is a value, so the budget always runs out

(fix f(f). f) 3 returns 3: the parameter shadows the function name

```
Z = lam g. (lam x. g (lam v. x x v)) (lam x. g (lam v. x x v))
```

Thunks for if, and, or

C[if c then t else f] = (C[c] (lam k_1. C[t]) (lam k_2. C[f])) I

Both branches become lambdas, so call-by-value does not enter them

The boolean picks one thunk; applying it to I runs only that branch

and and or use the same delay

if true then 1 else LOOP gives 1. (lam x. 0) LOOP exhausts

Thunk binders are k_1, k_2, ... chosen to avoid every name they wrap

What the compiler emits

Source: true and false

```
((((lam t. (lam f. t))
    (lam k_1. (lam t. (lam f. f))))
    (lam k_2. (lam t. (lam f. f))))
  (lam w. w))
```

TRUE chooses the first thunk, k_1, which holds FALSE

lam w. w forces it. 3 execution steps, decoded false

Every name here is expanded. No TRUE, no AND, no source nodes

Factorial of 4

fix fact(n). if n == 0 then 1 else n * fact (n - 1), applied to 4

Compiles without running. Pure term: demo-out/12.lam, 1,557 characters

Execution: 471 beta-value steps, result is a lambda

Observation: 333 beta-value steps, decoded 24

Source side, same evaluator: 24

Fibonacci of 5: 3,080 execution steps, 35 observation steps, decoded 5

Results from python MySolution/demo.py

Program	Exec steps	Obs steps	Decoded	Expected
2 + 3 * 4	4	26	14	14
2 - 5	87	2	0	0
if 2 <= 3 then 7 else 9	77	9	7	7
let x = 3 in (lam y. x + y) 4	4	13	7	7
fst / snd (pair 2 true)	6 / 6	4 / 2	2 / true	2 / true
(lam f. f 3) (lam x. x + 1)	4	10	4	4
true and false / false or true	3 / 3	2 / 2	false / true	false / true
if true then 1 else LOOP	3	3	1	1
(lam x. 0) LOOP	200	—	exhausted	exhausted
factorial 4	471	333	24	24
fibonacci 5	3080	35	5	5
LOOP	200	—	exhausted	exhausted

Testing

40 unittest tests, grouped by the seven testing items in the brief

Capture and shadowing, budget boundaries, left-to-right traces

Every operator with written-out answers, including 0 - 1 and equal operands

Fix binders named like Z's own names, and source names like s, z, k_1

Skipped and forced divergence; rejection in dead code

Compiler and evaluator each stubbed out to prove they stay separate

Evidence, not proof: exhaustion at 200 does not prove LOOP diverges

Two design decisions

Decision 1. One visitor evaluates both languages; the compiler cannot call it

- Gain: one application rule, so source and target cannot drift apart
- Cost: two substitution functions, and arg aliases on the target nodes

Decision 2. Decoding is a second reduction, not a reducer that knows numerals

- Gain: the reducer stays a pure beta engine
- Cost: two step counts, because a lambda can still hide work

Limits

No parser, no type checker, no web page

Naturals, booleans, and finite pairs only

Negative literals and unknown operators rejected everywhere; other type errors are not checked

Target execution needs closed terms; compilation accepts open ones

Substitution-based reduction is slow on big numerals; very deep terms can hit the Python stack